

Глава 7

Изменение данных

Эта глава будет посвящена операциям изменения данных: вставке новых строк в таблицы, обновлению уже существующих строк и их удалению. С простыми приемами использования команд INSERT, UPDATE и DELETE, предназначенных для выполнения указанных операций, вы уже познакомились, поэтому мы расскажем о некоторых более интересных способах применения этих команд.

7.1. Вставка строк в таблицы

Для работы нам потребуется создать еще две таблицы в базе данных «Авиаперевозки» (demo). Мы будем создавать их как временные таблицы, которые будут удаляться при отключении от базы данных. Использование временных таблиц позволит нам проводить эксперименты, будучи уверенными в том, что данные в постоянных таблицах модифицированы не будут, поэтому все запросы, которые вы выполняли ранее, будут работать так, как и работали.

Итак, создадим две копии таблицы «Самолеты» (aircrafts). Первая таблица-копия предназначена для хранения данных, взятых из таблицы-прототипа, а вторая таблица-копия будет использоваться в качестве журнальной таблицы: будем записывать в нее все операции, проведенные с первой таблицей.

Создадим первую таблицу, причем копировать данные из постоянной таблицы aircrafts не будем, о чем говорит предложение WITH NO DATA. Если бы мы решили скопировать в новую таблицу и все строки, содержащиеся в таблице-прототипе, тогда в команде CREATE TABLE мы могли бы использовать предложение WITH DATA или вообще не указывать его: по умолчанию строки копируются в создаваемую таблицу.

```
CREATE TEMP TABLE aircrafts_tmp AS  
SELECT * FROM aircrafts WITH NO DATA;
```

Наложим на таблицу необходимые ограничения: они не создаются при копировании таблицы. При массовом вводе данных гораздо более эффективным с точки зрения производительности было бы сначала добавить строки в таблицу, а уже потом накладывать ограничения на нее. Однако в нашем случае речь о массовом вводе не идет,

поэтому мы начнем с наложения ограничений, а уже потом добавим строки в таблицу.

```
ALTER TABLE aircrafts_tmp  
ADD PRIMARY KEY ( aircraft_code );
```

```
ALTER TABLE aircrafts_tmp  
ADD UNIQUE ( model );
```

Теперь создадим вторую таблицу, и также не будем копировать в нее данные из постоянной таблицы `aircrafts`.

```
CREATE TEMP TABLE aircrafts_log AS  
SELECT * FROM aircrafts WITH NO DATA;
```

Ограничения в виде первичного и уникального ключей этой таблице не требуются, но потребуются еще два столбца: первый будет содержать дату/время выполнения операции над таблицей `aircrafts_tmp`, а второй — наименование этой операции (`INSERT`, `UPDATE` или `DELETE`).

```
ALTER TABLE aircrafts_log  
ADD COLUMN when_add timestamp;
```

```
ALTER TABLE aircrafts_log  
ADD COLUMN operation text;
```

Поскольку в рассматриваемой ситуации копировать данные из постоянных таблиц во временные не требуется, то в качестве альтернативного способа создания временных таблиц можно было бы воспользоваться командой `CREATE TEMP TABLE` с предложением `LIKE`. Например:

```
CREATE TEMP TABLE aircrafts_tmp  
( LIKE aircrafts INCLUDING CONSTRAINTS INCLUDING INDEXES );
```

Но так как уникального индекса по столбцу `model` в таблице `aircrafts` нет, то для временной таблицы его пришлось бы сформировать с помощью команды `ALTER TABLE`, как и при использовании первого способа ее создания. Добавим, что предложение `LIKE` можно применять для создания не только временных таблиц, но и постоянных.

Поскольку у нас есть журнальная таблица `aircrafts_log`, мы можем записывать в нее все операции с таблицей `aircrafts_tmp`, т. е. вести историю изменений данных таблицы `aircrafts_tmp`.

Начнем работу с того, что скопируем в таблицу `aircrafts_tmp` все данные из таблицы `aircrafts`. Для выполнения не только «полезной» работы, но и ведения журнала изменений мы используем команду **INSERT с общим табличным выражением**.

Вообще, при классическом подходе для ведения учета изменений, внесенных в таблицы, используют триггеры или правила (rules), но их рассмотрение выходит за рамки этого пособия. Поэтому наш пример нужно рассматривать как иллюстрацию возможностей общих табличных выражений (CTE), а не как единственно верный подход.

```
WITH add_row AS
( INSERT INTO aircrafts_tmp
  SELECT * FROM aircrafts
  RETURNING *
)
INSERT INTO aircrafts_log
  SELECT add_row.aircraft_code, add_row.model, add_row.range,
         current_timestamp, 'INSERT'
  FROM add_row;
```

```
INSERT 0 9
```

Давайте рассмотрим эту команду более подробно. Обратите внимание, что вся «полезная» работа выполняется в рамках конструкции `WITH add_row AS (...)`. Здесь строки с помощью команды `SELECT` выбираются из таблицы `aircrafts` и вставляются в таблицу `aircrafts_tmp`. При вставке строк, выбранных из одной таблицы, в другую таблицу необходимо, чтобы число атрибутов и их типы данных во вставляемых строках были согласованы с числом столбцов и их типами данных в целевой таблице. Завершается конструкция `WITH add_row AS (...)` предложением `RETURNING *`, которое просто возвращает внешнему запросу все строки, успешно добавленные в таблицу `aircrafts_tmp`. Конечно же, при этом из таблицы `aircrafts_tmp` добавленные строки никуда не исчезают. Запрос получает имя `add_row`, на которое может ссылаться внешний запрос, когда он «хочет» обратиться к строкам, возвращенным с помощью предложения `RETURNING *`.

Теперь обратимся к внешнему запросу. В нем также присутствует команда `INSERT`, которая получает данные для ввода в таблицу `aircrafts_log` от запроса `SELECT`. Этот запрос, в свою очередь, получает данные от временной таблицы `add_row`, указанной в предложении `FROM`. Поскольку в предложении `RETURNING` мы указали в качестве возвращаемого значения `*`, то будут возвращены все столбцы таблицы `aircrafts_tmp`, т. е. той таблицы, в которую строки были вставлены. Следовательно, в команде `SELECT` внешнего запроса можно ссылаться на имена этих столбцов:

```
SELECT add_row.aircraft_code, add_row.model, add_row.range, ...
```

Поскольку в таблице `aircrafts_log` существует еще два столбца, то для них мы дополнительно передаем значения `current_timestamp` и `'INSERT'`.

Проверим, что получилось:

```
SELECT * FROM aircrafts_tmp ORDER BY model;
```

aircraft_code	model	range
319	Airbus A319-100	6700
320	Airbus A320-200	5700
321	Airbus A321-200	5600
733	Boeing 737-300	4200
763	Boeing 767-300	7900
773	Boeing 777-300	11100
CR2	Bombardier CRJ-200	2700
CN1	Cessna 208 Caravan	1200
SU9	Sukhoi SuperJet-100	3000

(9 строк)

Проверим также и содержимое журнальной таблицы:

```
SELECT * FROM aircrafts_log ORDER BY model;
```

-[RECORD 1]--+		
aircraft_code		319
model		Airbus A319-100
range		6700
when_add		2017-01-31 18:28:49.230179
operation		INSERT
-[RECORD 2]--+		
aircraft_code		320
model		Airbus A320-200
range		5700
when_add		2017-01-31 18:28:49.230179
operation		INSERT
...		

При вставке новых строк могут возникать ситуации, когда нарушается ограничение первичного или уникального ключей, поскольку вставляемые строки могут иметь значения ключевых атрибутов, совпадающие с теми, что уже имеются в таблице. Для таких случаев предусмотрено специальное средство — предложение `ON CONFLICT`,

оно предусматривает два варианта действий на выбор программиста. Первый вариант — отменять добавление новой строки, для которой имеет место конфликт значений ключевых атрибутов, и при этом не порождать сообщения об ошибке. Вторым вариантом является замена операции добавления новой строки операцией обновления существующей строки, с которой конфликтует добавляемая строка.

Начнем с первого варианта. Попробуем добавить строку, которая гарантированно будет конфликтовать с уже существующей строкой, причем как по первичному ключу `aircraft_code`, так и по уникальному ключу `model`.

```
WITH add_row AS
( INSERT INTO aircrafts_tmp
  VALUES ( 'SU9', 'Sukhoi SuperJet-100', 3000 )
  ON CONFLICT DO NOTHING
  RETURNING *
)
INSERT INTO aircrafts_log
  SELECT add_row.aircraft_code, add_row.model, add_row.range,
         current_timestamp, 'INSERT'
  FROM add_row;
```

Обратите внимание, что не будет выведено никаких сообщений об ошибках, как это и предполагалось. Строка добавлена не будет:

```
INSERT 0 0
```

Нужно учитывать, что сообщение о нуле строк относится к таблице `aircrafts_log`, т. е. к команде в главном запросе, а не в общем табличном выражении, в котором мы работаем с таблицей `aircrafts_tmp`. Проверьте, не была ли добавлена строка в таблицу `aircrafts_tmp`.

В том случае, когда в предложении `ON CONFLICT` не указана дополнительная информация об именах столбцов или ограничений, по которым предполагается возможный конфликт, проверка выполняется по первичному ключу и по уникальным ключам.

Укажем конкретный столбец для проверки конфликтующих значений. Пусть это будет `aircraft_code`, т. е. первичный ключ. Для упрощения команды не будем использовать общее табличное выражение. Добавляемая строка будет конфликтовать с существующей строкой как по столбцу `aircraft_code`, так и по столбцу `model`.

```
INSERT INTO aircrafts_tmp
  VALUES ( 'SU9', 'Sukhoi SuperJet-100', 3000 )
  ON CONFLICT ( aircraft_code ) DO NOTHING
  RETURNING *;
```

Получим только такое сообщение:

```
aircraft_code | model | range
-----+-----+-----
(0 строк)
```

```
INSERT 0 0
```

Сообщение было выведено потому, что в команду включено предложение RETURNING *. Сообщение о дублировании значений столбца model не выводится.

Давайте в команде INSERT изменим значение столбца aircraft_code, чтобы оно стало уникальным:

```
INSERT INTO aircrafts_tmp
VALUES ( 'S99', 'Sukhoi SuperJet-100', 3000 )
ON CONFLICT ( aircraft_code ) DO NOTHING
RETURNING *;
```

Поскольку конфликта по столбцу aircraft_code нет, то далее проверяется выполнение требования уникальности по столбцу model. В результате мы получим традиционное сообщение об ошибке, относящееся к столбцу model:

```
ОШИБКА: повторяющееся значение ключа нарушает ограничение уникальности
"aircrafts_tmp_model_key"
ПОДРОБНОСТИ: Ключ "(model)=(Sukhoi SuperJet-100)" уже существует.
```

Теперь рассмотрим второй вариант обработки предложения ON CONFLICT, когда операция вставки новой строки заменяется операцией обновления существующей строки, с которой и возник конфликт значений столбцов. Для реализации этой возможности служит предложение DO UPDATE.

Давайте модифицируем команду и добавим предложение DO UPDATE. Выберем такую политику для работы с таблицей aircrafts_tmp: если при вставке новой строки имеет место дублирование по атрибутам первичного ключа со строкой, находящейся в таблице, тогда мы будем обновлять значения всех остальных атрибутов в этой строке, независимо от того, совпадают ли они со значениями в новой строке или нет. В качестве примера сделаем так: в добавляемой строке значение атрибута model сделаем отличающимся от того, которое уже есть в таблице (вместо Sukhoi SuperJet-100 будет Sukhoi SuperJet), а значение атрибута range оставим без изменений (3000).

Внесем еще одно изменение: вместо имени столбца, образующего первичный ключ, с помощью предложения ON CONSTRAINT укажем наименование ограничения первичного ключа. Вот так выглядит команда с предложением DO UPDATE:

```

INSERT INTO aircrafts_tmp
VALUES ( 'SU9', 'Sukhoi SuperJet', 3000 )
ON CONFLICT ON CONSTRAINT aircrafts_tmp_pkey
DO UPDATE SET model = excluded.model,
               range = excluded.range
RETURNING *;

```

Поскольку мы включили в команду предложение RETURNING *, то СУБД сообщит о том, какие значения получают атрибуты обновленной строки. Как и планировалось, изменилось только значение атрибута model.

aircraft_code	model	range
SU9	Sukhoi SuperJet	3000

(1 строка)

В случае конфликта по столбцу aircraft_code будет обновлена та строка в таблице aircrafts_tmp, с которой конфликтовала вновь добавляемая строка. В результате новая строка добавлена не будет, а будет обновлено значение столбца model в строке, уже находящейся в таблице. А где PostgreSQL возьмет значение для использования в команде UPDATE? Это значение будет взято из специальной таблицы excluded, которая поддерживается самой СУБД. В этой таблице хранятся все строки, предлагаемые для вставки в рамках текущей команды INSERT. Вот это значение — excluded.model. Значение столбца range также будет обновлено, но его новое значение — excluded.range — совпадает со старым.

Обратите внимание, что в предложении DO UPDATE не указывается имя таблицы, т. к. таблица будет та же самая, которая указана в предложении INSERT.

Предложение ON CONFLICT DO UPDATE гарантирует атомарное выполнение операции вставки или обновления строк. Атомарность означает, что проверка наличия конфликта и последующее обновление выполняются как неделимая операция, т. е. другие транзакции не могут изменить значение столбца, вызывающее конфликт, так, чтобы в результате конфликт исчез и уже стало возможным выполнить операцию INSERT, а не UPDATE, или, наоборот, в случае отсутствия конфликта он вдруг появился, и уже операция INSERT стала бы невозможной. Такая атомарная операция даже имеет название UPSERT — «UPDATE или INSERT».

Для массового ввода строк в таблицы используется команда COPY. Эта команда может копировать данные из файла в таблицу. Причем, в качестве файла может служить и стандартный ввод. Хотя в этом разделе пособия мы, в основном, говорим о вставке строк в таблицы, но нужно сказать и о том, что эта команда может также копировать данные из таблиц в файлы и на стандартный вывод.

В качестве примера ввода данных из файла давайте добавим две строки в таблицу `aircrafts_tmp`. Сначала необходимо подготовить текстовый файл, содержащий новые данные. В этом файле каждая строка соответствует одной строке таблицы. Значения атрибутов разделяются символами табуляции, поэтому пробелы, которые есть в столбце `model`, можно вводить в файл без каких-либо дополнительных экранирующих символов. Заключать строковые значения в одинарные кавычки не нужно, иначе они также будут введены в таблицу. Завершить файл нужно строкой, содержащей только символы «\.».

```
IL9      Ilyushin IL96      9800
I93      Ilyushin IL96-300  9800
\.
```

Теперь нужно ввести команду `COPY`, указав полный путь к вашему файлу:

```
COPY aircrafts_tmp FROM '/home/postgres/aircrafts.txt';
```

В результате будет выведено сообщение об успешном добавлении двух строк:

```
COPY 2
```

Давайте проверим, что получилось:

```
SELECT * FROM aircrafts_tmp;
```

Вы увидите, что новые строки были добавлены, но все те, что уже находились в таблице, удалены не были.

При использовании команды `COPY` выполняются проверки всех ограничений, наложенных на таблицу, поэтому ввести дублирующие данные не получится.

Эту команду можно использовать и для вывода данных из таблицы в файл:

```
COPY aircrafts_tmp TO '/home/postgres/aircrafts_tmp.txt'
WITH ( FORMAT csv );
```

Предложение `FORMAT csv` говорит о том, что при выводе данных значения столбцов разделяются запятыми (CSV — Comma Separated Values). Получим файл такого вида:

```
773,Boeing 777-300,11100
763,Boeing 767-300,7900
SU9,Sukhoi SuperJet-100,3000
...
```

Если формат не указывать, то данные будут выведены с использованием символов табуляции в качестве разделителей значений атрибутов.

7.2. Обновление строк в таблицах

Команда UPDATE предназначена для обновления данных в таблицах. Начнем с того, что покажем, как и при изучении команды INSERT, как можно организовать запись выполненных операций в журнальную таблицу. Эта команда аналогична команде, уже рассмотренной в предыдущем разделе. В ней также «полезная» работа выполняется в общем табличном выражении, а запись в журнальную таблицу — в основном запросе.

```
WITH update_row AS
( UPDATE aircrafts_tmp
  SET range = range * 1.2
  WHERE model ~ '^Bom'
  RETURNING *
)
INSERT INTO aircrafts_log
  SELECT ur.aircraft_code, ur.model, ur.range,
         current_timestamp, 'UPDATE'
  FROM update_row ur;
```

Выполнив команду, в ответ получим сообщение

```
INSERT 0 1
```

Напомним, что выведенное сообщение относится непосредственно к внешнему запросу, в котором выполняется операция INSERT, добавляющая строку в журнальную таблицу. Конечно, если бы строка в таблице aircrafts_tmp не была успешно обновлена, тогда предложение RETURNING * не возвратило бы внешнему запросу ни одной строки, и, следовательно, тогда просто не было бы данных для формирования новой строки в таблице aircrafts_log.

При использовании команды UPDATE в общем табличном выражении нужно учитывать, что главный запрос может получить доступ к обновленным данным *только через временную таблицу*, которую формирует предложение RETURNING:

```
...
FROM update_row ur;
```

Можно выполнить выборку из журнальной таблицы aircrafts_log, чтобы посмотреть — правда, не очень длинную — историю изменений строки с описанием самолета Bombardier CRJ-200.

```
SELECT * FROM aircrafts_log
WHERE model ~ '^Bom' ORDER BY when_add;
```

```
-[ RECORD 1 ]--+-+-----
aircraft_code | CR2
model         | Bombardier CRJ-200
range        | 2700
when_add      | 2017-02-05 00:27:38.591958
operation     | INSERT
-[ RECORD 2 ]--+-+-----
aircraft_code | CR2
model         | Bombardier CRJ-200
range        | 3240
when_add      | 2017-02-05 00:27:56.688933
operation     | UPDATE
```

Представим себе такую ситуацию: руководство компании хочет видеть динамику продаж билетов по всем направлениям, а именно: общее число проданных билетов и дату/время последнего увеличения их числа для конкретного направления.

Создадим временную таблицу `tickets_directions` с четырьмя столбцами:

- города отправления и прибытия — `departure_city` и `arrival_city`;
- дата/время последнего увеличения числа проданных билетов — `last_ticket_time`;
- число проданных билетов на этот момент времени по данному направлению — `tickets_num`.

Создадим таблицу с помощью запроса к представлению «Маршруты» и заполним данными, однако в ней сначала будет только два первых столбца.

```
CREATE TEMP TABLE tickets_directions AS
SELECT DISTINCT departure_city, arrival_city FROM routes;
```

Ключевое слово `DISTINCT` является здесь обязательным: ведь нам нужны только уникальные пары городов отправления и прибытия.

Добавим еще два столбца и заполним столбец-счетчик нулевыми значениями.

```
ALTER TABLE tickets_directions
ADD COLUMN last_ticket_time timestamp;

ALTER TABLE tickets_directions
ADD COLUMN tickets_num integer DEFAULT 0;
```

Поскольку PostgreSQL не требует обязательного создания первичного ключа, то не будем создавать его. Это не мешает нам однозначно идентифицировать строки в таблице `tickets_directions`.

Поскольку в команде `ALTER TABLE` нет предложения `WHERE`, в котором было бы условие, ограничивающее множество обновляемых строк, то будут обновлены все строки таблицы — во все будет записано значение 0 в столбец `tickets_num`.

Для того чтобы не усложнять изложение материала, создадим временную таблицу, являющуюся аналогом таблицы «Перелеты», однако без внешних ключей. Поэтому мы сможем добавлять в нее строки, не заботясь о добавлении строк в таблицы «Билеты» и «Бронирования». Тем не менее первичный ключ все же создадим, чтобы продемонстрировать, что в случае попытки ввода строк с дубликатными значениями первичного ключа значения счетчиков в таблице `tickets_directions` наращиваться не будут.

```
CREATE TEMP TABLE ticket_flights_tmp AS
  SELECT * FROM ticket_flights WITH NO DATA;
```

```
ALTER TABLE ticket_flights_tmp
  ADD PRIMARY KEY ( ticket_no, flight_id );
```

Теперь представим команду, которая и будет добавлять новую запись о продаже билета и увеличивать в таблице `tickets_directions` значение счетчика проданных билетов.

```
WITH sell_ticket AS
( INSERT INTO ticket_flights_tmp
  ( ticket_no, flight_id, fare_conditions, amount )
  VALUES ( '1234567890123', 30829, 'Economy', 12800 )
  RETURNING *
)
UPDATE tickets_directions td
  SET last_ticket_time = current_timestamp,
      tickets_num = tickets_num + 1
  WHERE ( td.departure_city, td.arrival_city ) =
    ( SELECT departure_city, arrival_city
      FROM flights_v
      WHERE flight_id = ( SELECT flight_id FROM sell_ticket )
    );
```

```
UPDATE 1
```

Этот запрос работает следующим образом. Добавление новой записи о бронировании авиаперелета производится в общем табличном выражении, а наращивание соответствующего счетчика — в главном запросе. Поскольку в общем табличном выражении присутствует предложение `RETURNING *`, значения атрибутов добавленной строки будут доступны в главном запросе посредством обращения к временной таблице `sell_ticket`. Конечно, если строка фактически не будет добавлена из-за дублирования значения первичного ключа, тогда будет сгенерировано сообщение об ошибке, в результате главный запрос выполнен не будет, следовательно, таблица `tickets_directions` не будет обновлена.

В главном запросе мы обновляем всего два атрибута, причем значение атрибута `tickets_num` может увеличиться только на единицу, поскольку мы добавляем одну строку в таблицу `ticket_flights_tmp`. Остается выяснить, каким образом можно определить ту строку в таблице `tickets_directions`, атрибуты которой нужно обновить. Нам требуется на основе значения идентификатора рейса `flight_id`, на который был забронирован билет (перелет), определить города отправления и прибытия, которые как раз и идентифицируют строку в таблице `tickets_directions`. Эти три атрибута присутствуют в представлении `flights_v`. Подзапрос обращается к этому представлению, а вложенный подзапрос возвращает значение идентификатора рейса `flight_id`, на который был забронирован билет (перелет). Назначение вложенного подзапроса в том, чтобы в условии `WHERE flight_id = ...` не дублировать значение атрибута `flight_id`, использованное в команде `INSERT` (в данном примере это 30829). Тем самым должен быть снижен риск ошибки при вводе данных.

Обратите внимание, что подзапрос в предложении `WHERE` возвращает два столбца, и сравнение выполняется также сразу с двумя столбцами.

Посмотрим, что получилось:

```
SELECT *
  FROM tickets_directions
 WHERE tickets_num > 0;
```

```
-[ RECORD 1 ]-----+-----
departure_city  | Сочи
arrival_city    | Красноярск
last_ticket_time | 2017-02-04 21:15:32.903687
tickets_num     | 1
```

Представим другой вариант этой команды. Его принципиальное отличие от первого варианта состоит в том, что для определения обновляемой строки в таблице

`tickets_directions` используется **операция соединения таблиц**. Здесь в главном запросе UPDATE присутствует предложение FROM, однако в этом предложении указывается только представление `flights_v`, а таблицу `tickets_directions` в предложение FROM включать не нужно, хотя она и участвует в выполнении соединения таблиц. Конечно, в предложении SET присваивать новые значения можно только атрибутам таблицы `tickets_directions`, поскольку именно она приведена в предложении UPDATE.

```
WITH sell_ticket AS
( INSERT INTO ticket_flights_tmp
  (ticket_no, flight_id, fare_conditions, amount )
  VALUES ( '1234567890123', 7757, 'Economy', 3400 )
  RETURNING *
)
UPDATE tickets_directions td
  SET last_ticket_time = current_timestamp,
      tickets_num = tickets_num + 1
  FROM flights_v f
 WHERE td.departure_city = f.departure_city
       AND td.arrival_city = f.arrival_city
       AND f.flight_id = ( SELECT flight_id FROM sell_ticket );
```

UPDATE 1

Посмотрим, что получилось:

```
SELECT *
  FROM tickets_directions
 WHERE tickets_num > 0;
```

```
--[ RECORD 1 ]-----+-----
departure_city | Сочи
arrival_city   | Красноярск
last_ticket_time | 2017-02-04 21:15:32.903687
tickets_num    | 1
--[ RECORD 2 ]-----+-----
departure_city | Москва
arrival_city   | Сочи
last_ticket_time | 2017-02-04 21:18:40.353408
tickets_num    | 1
```

Чтобы увидеть комбинированную строку, которая получилась при соединении таблиц `tickets_directions` и `flights_v`, можно включить в команду UPDATE предложение RETURNING *.

7.3. Удаление строк из таблиц

Начнем рассмотрение команды DELETE, предназначенной для удаления данных из таблиц, с того, что, как и при изучении команды INSERT, покажем, как можно организовать запись выполненных операций в журнальную таблицу. Эта команда аналогична команде, уже рассмотренной в предыдущем разделе. В ней также «полезная» работа выполняется в общем табличном выражении, а запись в журнальную таблицу — в основном запросе.

```
WITH delete_row AS
( DELETE FROM aircrafts_tmp
  WHERE model ~ '^Bom'
  RETURNING *
)
INSERT INTO aircrafts_log
  SELECT dr.aircraft_code, dr.model, dr.range,
         current_timestamp, 'DELETE'
  FROM delete_row dr;
```

Выполнив команду, в ответ получим сообщение

```
INSERT 0 1
```

Напомним, что выведенное сообщение относится непосредственно к внешнему запросу, в котором выполняется операция INSERT, добавляющая строку в журнальную таблицу.

Посмотрим историю изменений строки с описанием самолета Bombardier CRJ-200:

```
SELECT * FROM aircrafts_log
  WHERE model ~ '^Bom' ORDER BY when_add;
```

-[RECORD 1]--+	
aircraft_code	CR2
model	Bombardier CRJ-200
range	2700
when_add	2017-02-05 00:27:38.591958
operation	INSERT
-[RECORD 2]--+	
aircraft_code	CR2
model	Bombardier CRJ-200
range	3240
when_add	2017-02-05 00:27:56.688933
operation	UPDATE

```

-[ RECORD 3 ]--+-----
aircraft_code | CR2
model         | Bombardier CRJ-200
range         | 3240
when_add      | 2017-02-05 00:34:59.510911
operation     | DELETE

```

Для удаления конкретных строк из данной таблицы можно использовать информацию не только из нее, но также и из других таблиц. Выбирать строки для удаления можно двумя способами: использовать подзапросы к этим таблицам в предложении WHERE или указать дополнительные таблицы в предложении USING, а затем в предложении WHERE записать условия соединения таблиц. Поскольку первый способ является традиционным, то мы покажем второй из них.

Предположим, что руководство авиакомпании решило удалить из парка самолетов машины компаний Boeing и Airbus, имеющие наименьшую дальность полета.

Решим эту задачу следующим образом. В общем табличном выражении с помощью условия `model ~ '^Airbus' OR model ~ '^Boeing'` в предложении WHERE отберем модели только компаний Boeing и Airbus. Затем воспользуемся оконной функцией `rank` и произведем ранжирование моделей каждой компании по возрастанию дальности полета. Те модели, ранг которых окажется равным 1, будут иметь наименьшую дальность полета.

В предложении USING сформируем соединение таблицы `aircrafts_tmp` с временной таблицей `min_ranges`, а затем в предложении WHERE зададим условия для отбора строк.

```

WITH min_ranges AS
( SELECT aircraft_code,
        rank() OVER (
            PARTITION BY left( model, 6 )
            ORDER BY range
        ) AS rank
  FROM aircrafts_tmp
 WHERE model ~ '^Airbus' OR model ~ '^Boeing'
)
DELETE FROM aircrafts_tmp a
  USING min_ranges mr
 WHERE a.aircraft_code = mr.aircraft_code
    AND mr.rank = 1
RETURNING *;

```

Мы включили в команду DELETE предложение RETURNING * для того, чтобы показать, как выглядят комбинированные строки, сформированные с помощью предложения USING. Конечно, удаляются не они, а только оригинальные строки из таблицы aircrafts_tmp.

aircraft_code	model	range	aircraft_code	rank
321	Airbus A321-200	5600	321	1
733	Boeing 737-300	4200	733	1

(2 строки)

В заключение этого раздела упомянем еще команду TRUNCATE, которая позволяет быстро удалить все строки из таблицы. Следующие две команды позволяют удалить все строки из таблицы aircrafts_tmp:

```
DELETE FROM aircrafts_tmp;  
TRUNCATE aircrafts_tmp;
```

Однако команда TRUNCATE работает быстрее.

Контрольные вопросы и задания

1. Добавьте в определение таблицы aircrafts_log значение по умолчанию current_timestamp и соответствующим образом измените команды INSERT, приведенные в тексте главы.
2. В предложении RETURNING можно указывать не только символ «*», означающий выбор всех столбцов таблицы, но и более сложные выражения, сформированные на основе этих столбцов. В тексте главы мы копировали содержимое таблицы «Самолеты» в таблицу aircrafts_tmp, используя в предложении RETURNING именно «*». Однако возможен и другой вариант запроса:

```
WITH add_row AS  
( INSERT INTO aircrafts_tmp  
  SELECT * FROM aircrafts  
  RETURNING aircraft_code, model, range,  
            current_timestamp, 'INSERT'  
)  
INSERT INTO aircrafts_log  
  SELECT ? FROM add_row;
```

Что нужно написать в этом запросе вместо вопросительного знака?

3. Если бы мы для копирования данных в таблицу `aircrafts_tmp` использовали команду `INSERT` без общего табличного выражения

```
INSERT INTO aircrafts_tmp SELECT * FROM aircrafts;
```

то в качестве выходного результата мы увидели бы сообщение

```
INSERT 0 9
```

Как вы думаете, что будет выведено, если дополнить команду предложением `RETURNING *`?

```
INSERT INTO aircrafts_tmp SELECT * FROM aircrafts RETURNING *;
```

Проверьте ваши предположения на практике. Подумайте, каким образом можно использовать выведенный результат?

4. В тексте главы в предложениях `ON CONFLICT` команды `INSERT` мы использовали только выражения, состоящие из имени одного столбца. Однако в таблице «Места» (`seats`) первичный ключ является составным и включает два столбца.

Напишите команду `INSERT` для вставки новой строки в эту таблицу и предусмотрите возможный конфликт добавляемой строки со строкой, уже имеющейся в таблице. Сделайте два варианта предложения `ON CONFLICT`: первый — с использованием перечисления имен столбцов для проверки наличия дублирования, второй — с использованием предложения `ON CONSTRAINT`.

Для того чтобы не изменить содержимое таблицы «Места», создайте ее копию и выполняйте все эти эксперименты с таблицей-копией.

5. В предложении `DO UPDATE` команды `INSERT` может использоваться и условие `WHERE`. Самостоятельно ознакомьтесь с этой возможностью с помощью документации и напишите такую команду `INSERT`.
6. Команда `COPY` по умолчанию ожидает получения вводимых данных в формате `text`, когда значения данных разделяются символами табуляции. Однако можно представлять входные данные в формате `CSV` (`Comma Separated Values`), т. е. использовать в качестве разделителя запятую.

```
COPY aircrafts_tmp FROM STDIN WITH ( FORMAT csv );
```

Вводите данные для копирования, разделяя строки переводом строки. Закончите ввод строкой `'\.'`.

IL9, Ilyushin IL96, 9800
I93, Ilyushin IL96-300, 9800
\.

COPY 2

SELECT * FROM aircrafts_tmp;

aircraft_code	model	range
...		
CN1	Cessna 208 Caravan	1200
CR2	Bombardier CRJ-200	2700
IL9	Ilyushin IL96	9800
I93	Ilyushin IL96-300	9800

(11 строк)

Как вы думаете, почему при выводе данных из таблицы вновь введенные значения в столбце `model` оказались смещены вправо?

7. Команда `COPY` позволяет получить входные данные из файла и поместить их в таблицу. Этот файл должен быть доступен тому пользователю операционной системы, от имени которого запущен серверный процесс, как правило, это пользователь `postgres`.

Подготовьте файл, например, `/home/postgres/aircrafts_tmp.csv`, имеющий такую структуру:

- каждая строка файла соответствует одной строке таблицы `aircrafts_tmp`;
- значения данных в строке файла разделяются запятыми.

Например:

773,Boeing 777-300,11100
763,Boeing 767-300,7900
SU9,Sukhoi SuperJet-100,3000

Введите в этот файл данные о нескольких самолетах, причем часть из них уже должна быть представлена в таблице, а часть — нет.

Поскольку при выполнении команды `COPY` проверяются все ограничения целостности, наложенные на таблицу, то дублирующие строки добавлены, конечно же, не будут. А как вы думаете, строки, содержащиеся в этом же файле, но отсутствующие в таблице, будут добавлены или нет?

Проверьте свою гипотезу, выполнив вставку строк в таблицу из этого файла:

```
COPY aircrafts_tmp  
FROM '/home/postgres/aircrafts_tmp.csv' WITH ( FORMAT csv );
```

- 8.* В тексте главы был приведен запрос, предназначенный для учета числа билетов, проданных по всем направлениям на текущую дату. Однако тот запрос был рассчитан на одновременное добавление только одной записи в таблицу «Перелеты» (ticket_flights_tmp). Ниже мы предложим более универсальный запрос, который предусматривает возможность единовременного ввода нескольких записей о перелетах, выполняемых на различных рейсах.

Для проверки работоспособности предлагаемого запроса выберем несколько рейсов по маршрутам: Красноярск — Москва, Москва — Сочи, Сочи — Москва, Сочи — Красноярск. Для определения идентификаторов рейсов сформируем вспомогательный запрос, в котором даты начала и конца рассматриваемого периода времени зададим с помощью функции `bookings.now`. Использование этой функции необходимо, поскольку в будущих версиях базы данных могут быть представлены другие диапазоны дат.

```
SELECT flight_no, flight_id, departure_city,  
       arrival_city, scheduled_departure  
FROM flights_v  
WHERE scheduled_departure  
       BETWEEN bookings.now() AND bookings.now() + INTERVAL '15 days'  
AND ( departure_city, arrival_city ) IN  
    ( ( 'Красноярск', 'Москва' ),  
      ( 'Москва', 'Сочи' ),  
      ( 'Сочи', 'Москва' ),  
      ( 'Сочи', 'Красноярск' )  
    )  
ORDER BY departure_city, arrival_city, scheduled_departure;
```

Обратите внимание на предикат `IN`: в нем используются не индивидуальные значения, а пары значений.

Предположим, что в течение указанного интервала времени пассажир планирует совершить перелеты по маршруту: Красноярск — Москва, Москва — Сочи, Сочи — Москва, Москва — Сочи, Сочи — Красноярск. Выполнив вспомогательный запрос, выберем следующие идентификаторы рейсов (в этом же порядке): 13829, 4728, 30523, 7757, 30829.

```

WITH sell_tickets AS
( INSERT INTO ticket_flights_tmp
  ( ticket_no, flight_id, fare_conditions, amount )
  VALUES ( '1234567890123', 13829, 'Economy', 10500 ),
           ( '1234567890123', 4728, 'Economy', 3400 ),
           ( '1234567890123', 30523, 'Economy', 3400 ),
           ( '1234567890123', 7757, 'Economy', 3400 ),
           ( '1234567890123', 30829, 'Economy', 12800 )
  RETURNING *
)
UPDATE tickets_directions td
  SET last_ticket_time = current_timestamp,
      tickets_num = tickets_num +
        ( SELECT count( * )
          FROM sell_tickets st, flights_v f
          WHERE st.flight_id = f.flight_id
              AND f.departure_city = td.departure_city
              AND f.arrival_city = td.arrival_city
        )
  WHERE ( td.departure_city, td.arrival_city ) IN
    ( SELECT departure_city, arrival_city
      FROM flights_v
      WHERE flight_id IN ( SELECT flight_id FROM sell_tickets )
    );

UPDATE 4

```

В этой версии запроса предусмотрен единовременный ввод нескольких строк в таблицу `ticket_flights_tmp`, причем перелеты могут выполняться на различных рейсах. Поэтому необходимо преобразовать список идентификаторов этих рейсов в множество пар «город отправления — город прибытия», поскольку именно для таких пар и ведется подсчет числа забронированных перелетов. Эта задача решается в предложении `WHERE`, где вложенный подзапрос формирует список идентификаторов рейсов, а внешний подзапрос преобразует этот список в множество пар «город отправления — город прибытия». Затем с помощью предиката `IN` производится отбор строк таблицы `tickets_directions` для обновления.

Теперь обратимся к предложению `SET`. Подзапрос с функцией `count` вычисляет количество перелетов по *каждому* направлению. Это коррелированный подзапрос: он выполняется для каждой строки, отобранной в предложении `WHERE`. В нем используется соединение временной таблицы `sell_tickets` с представлением `flights_v`. Это нужно для того, чтобы подсчитать все перелеты, соот-

ветствующие паре атрибутов «город отправления — город прибытия», взятых из текущей обновляемой строки таблицы `tickets_directions`. Этот подзапрос позволяет учесть такой факт: рейсы могут иметь различные идентификаторы `flight_id`, но при этом соответствовать одному и тому же направлению, а в таблице `tickets_directions` учитываются именно направления.

В случае попытки повторного бронирования одного и того же перелета для данного пассажира, т. е. ввода строки с дубликатом первичного ключа, такая строка будет отвергнута, и будет сгенерировано сообщение об ошибке. В таком случае и таблица `tickets_directions` не будет обновлена.

Давайте посмотрим, что изменилось в таблице `tickets_directions`.

```
SELECT departure_city AS dep_city,
       arrival_city AS arr_city,
       last_ticket_time,
       tickets_num AS num
FROM tickets_directions
WHERE tickets_num > 0
ORDER BY departure_city, arrival_city;
```

По маршруту Москва — Сочи наш пассажир приобретал два билета, что и отражено в выборке.

dep_city	arr_city	last_ticket_time	num
Красноярск	Москва	2017-02-04 14:02:23.769443	1
Москва	Сочи	2017-02-04 14:02:23.769443	2
Сочи	Красноярск	2017-02-04 14:02:23.769443	1
Сочи	Москва	2017-02-04 14:02:23.769443	1

(4 строки)

А это информация о каждом перелете, забронированном нашим пассажиром:

```
SELECT * FROM ticket_flights_tmp;
```

ticket_no	flight_id	fare_conditions	amount
1234567890123	13829	Economy	10500.00
1234567890123	4728	Economy	3400.00
1234567890123	30523	Economy	3400.00
1234567890123	7757	Economy	3400.00
1234567890123	30829	Economy	12800.00

(5 строк)

Задание. Модифицируйте запрос и таблицу `tickets_directions` так, чтобы учет числа забронированных перелетов по различным маршрутам выполнялся для каждого класса обслуживания: `Economy`, `Business` и `Comfort`.

- 9.* Предположим, что руководство нашей авиакомпании решило отказаться от использования самолетов компаний `Boeing` и `Airbus`, имеющих наименьшее количество пассажирских мест в салонах. Мы должны соответствующим образом откорректировать таблицу «Самолеты» (`aircrafts_tmp`).

Мы предлагаем такой алгоритм.

Шаг 1. Для каждой модели вычислить общее число мест в салоне.

Шаг 2. Используя оконную функцию `rank`, присвоить моделям ранги на основе числа мест (упорядочив их по возрастанию числа мест). Ранжирование выполняется *в пределах каждой компании-производителя*, т. е. для `Boeing` и для `Airbus` — отдельно. Ранг, равный 1, соответствует наименьшему числу мест.

Шаг 3. Выполнить удаление тех строк из таблицы `aircrafts_tmp`, которые удовлетворяют следующим требованиям: модель — `Boeing` или `Airbus`, а число мест в салоне — минимальное из всех моделей данной компании-производителя, т. е. модель имеет ранг, равный 1.

```
WITH aircrafts_seats AS
( SELECT aircraft_code, model, seats_num,
      rank() OVER (
        PARTITION BY left( model, strpos( model, ' ' ) - 1 )
        ORDER BY seats_num
      )
FROM
  ( SELECT a.aircraft_code, a.model, count( * ) AS seats_num
    FROM aircrafts_tmp a, seats s
    WHERE a.aircraft_code = s.aircraft_code
    GROUP BY 1, 2
  ) AS seats_numbers
)
DELETE FROM aircrafts_tmp a
USING aircrafts_seats a_s
WHERE a.aircraft_code = a_s.aircraft_code
      AND left( a.model, strpos( a.model, ' ' ) - 1 )
      IN ( 'Boeing', 'Airbus' )
      AND a_s.rank = 1
RETURNING *;
```

Шаг 1 выполняется в подзапросе в предложении WITH. Шаг 2 — в главном запросе в предложении WITH. Шаг 3 реализуется командой DELETE.

Обратите внимание, что название компании-производителя мы определяем путем взятия подстроки от значения атрибута model: от начала строки до пробельного символа (используем функции left и strpos). Мы включили предложение RETURNING *, чтобы увидеть, какие именно модели были удалены.

Предложение WITH выдает такой результат:

aircraft_code	model	seats_num	rank
319	Airbus A319-100	116	1
320	Airbus A320-200	140	2
321	Airbus A321-200	170	3
733	Boeing 737-300	130	1
763	Boeing 767-300	222	2
773	Boeing 777-300	402	3
CR2	Bombardier CRJ-200	50	1
CN1	Cessna 208 Caravan	12	1
SU9	Sukhoi SuperJet-100	97	1

(9 строк)

Очевидно, что должны быть удалены модели с кодами 319 и 733.

После выполнения запроса получим (это работает предложение RETURNING *):

```
-[ RECORD 1 ]--+-----
aircraft_code | 319
model         | Airbus A319-100
range         | 6700
aircraft_code | 319
model         | Airbus A319-100
seats_num     | 116
rank          | 1
-[ RECORD 2 ]--+-----
aircraft_code | 733
model         | Boeing 737-300
range         | 4200
aircraft_code | 733
model         | Boeing 737-300
seats_num     | 130
rank          | 1
```

DELETE 2

Обратите внимание, что в результате были выведены комбинированные строки, полученные при соединении таблицы `aircrafts_tmp` с временной таблицей `aircrafts_seats`, указанной в предложении `USING`. Но удалены были, конечно, строки из таблицы `aircrafts_tmp`.

Задание. Предложите другой вариант решения этой задачи. Например, можно поступить так: оставить предложение `WITH` без изменений, из команды `DELETE` убрать предложение `USING`, а в предложении `WHERE` вместо соединения таблиц использовать подзапрос с предикатом `IN` для получения списка кодов удаляемых моделей самолетов.

Еще один вариант решения задачи связан с использованием представлений, которые мы рассматривали в главе 5. Можно создать представление на основе таблиц «Самолеты» (`aircrafts`) и «Места» (`seats`) и перенести конструкцию с функциями `left` и `strpos` в представление. В нем будут вычисляемые столбцы: `company` — «Компания-производитель самолетов» и `seats_num` — «Число мест».

```
CREATE VIEW aircrafts_seats AS
( SELECT a.aircraft_code,
      a.model,
      left( a.model,
      strpos( a.model, ' ' ) - 1 ) AS company,
      count( * ) AS seats_num
  FROM aircrafts a, seats s
 WHERE a.aircraft_code = s.aircraft_code
 GROUP BY 1, 2, 3
 );
```

Имея это представление, можно использовать его в конструкции `WITH`. При этом вызов функции `rank` может упроститься:

```
rank() OVER ( PARTITION BY company ORDER BY seats_num )
```

Для выбора удаляемых строк в команде `DELETE` можно использовать, например, подзапрос в предикате `IN`. При этом не забывайте, что значение столбца `rank` для них будет равно 1.

Еще одна идея: для выбора минимальных значений числа мест в самолетах можно попытаться в качестве замены оконной функции `rank` использовать предложения `LIMIT 1` и `ORDER BY`. В таком случае не потребуются также и функция `min`.

- 10.* В реальной работе иногда возникают ситуации, когда требуется быстро заполнить таблицу тестовыми данными. В таком случае удобно воспользоваться командой INSERT с подзапросом. Конечно, число атрибутов и их типы данных в подзапросе SELECT должны быть такими, какие ожидает получить команда INSERT.

Продemonстрируем такой прием на примере таблицы «Места» (seats). Для того чтобы выполнить команду, приведенную в этом упражнении, нужно либо сначала удалить все строки из таблицы seats, чтобы можно было добавлять строки в эту таблицу

```
DELETE FROM seats;
```

либо создать копию этой таблицы

```
CREATE TABLE seats_tmp AS  
  SELECT * FROM seats;
```

чтобы работать с копией.

Итак, как сформировать тестовые данные автоматическим способом? Для этого сначала нужно подготовить исходные данные, на основе которых и будут формироваться результирующие значения для вставки в таблицу «Места».

В рамках реляционной модели наиболее естественным будет представление исходных данных в виде таблиц. Для формирования каждой строки таблицы «Места» нужно задать код модели самолета, класс обслуживания и номер места, который состоит из двух компонентов: номера ряда и буквенного идентификатора позиции в ряду.

Поскольку размеры и компоновки салонов различаются, необходимо для каждой модели указать предельное число рядов кресел в салонах бизнес-класса и экономического класса, а также число кресел в каждом ряду. Это число можно задать с помощью указания буквенного идентификатора для самого последнего кресла в ряду. Например, если в ряду всего шесть кресел, тогда их буквенные обозначения будут такими: A, B, C, D, E, F. Таким образом, последней будет буква F. В салоне бизнес-класса число мест в ряду меньше, чем в салоне экономического класса, но для упрощения задачи примем эти числа одинаковыми.

В результате получим первую исходную таблицу с атрибутами:

- код модели самолета;
- номер последнего ряда кресел в салоне бизнес-класса;

- номер последнего ряда кресел в салоне экономического класса;
- буква, обозначающая позицию последнего кресла в ряду.

Классы обслуживания также поместим в отдельную таблицу. В ней будет всего один атрибут — класс обслуживания.

Список номеров рядов также поместим в отдельную таблицу. В ней будет также всего один атрибут — номер ряда.

Так же поступим и с буквенными обозначениями кресел в ряду. В этой таблице будет один атрибут — латинская буква, обозначающая позицию кресла.

В принципе можно было бы создать все четыре таблицы с помощью команды `CREATE TABLE` и ввести в них исходные данные, а затем использовать эти таблицы в команде `SELECT`. Но команда `SELECT` позволяет использовать в предложении `FROM` виртуальные таблицы, которые можно создавать с помощью предложения `VALUES`. Для этого непосредственно в текст команды записываются группы значений, представляющие собой строки такой виртуальной таблицы. Каждая такая строка заключается в круглые скобки. Вся эта конструкция получает имя таблицы, и к ней прилагается список атрибутов. Это выглядит, например, следующим образом:

FROM

```
( VALUES ( 'SU9', 3, 20, 'F' ),
          ( '773', 5, 30, 'I' ),
          ( '763', 4, 25, 'H' ),
          ( '733', 3, 20, 'F' ),
          ( '320', 5, 25, 'F' ),
          ( '321', 4, 20, 'F' ),
          ( '319', 3, 20, 'F' ),
          ( 'CN1', 0, 10, 'B' ),
          ( 'CR2', 2, 15, 'D' )
    ) AS aircraft_info ( aircraft_code, max_seat_row_business,
                       max_seat_row_economy, max_letter )
```

Здесь `aircraft_info` определяет имя виртуальной таблицы, а список идентификаторов — имена ее атрибутов (`aircraft_code`, `max_seat_row_business`, `max_seat_row_economy`, `max_letter`). Эти атрибуты можно использовать во всех частях команды `SELECT`, как если бы это были атрибуты обычной таблицы.

Остальные виртуальные таблицы создаются аналогичным способом.

Для соединения таблиц используется ключевое слово CROSS JOIN, хотя в данном случае вместо этого можно было просто поставить запятые.

Как это и бывает всегда, четыре таблицы образуют декартово произведение из своих строк, а затем на основе условия WHERE «лишние» строки отбрасываются. В этом условии используется условный оператор CASE. Он позволяет нам поставить допустимый номер ряда в зависимость от класса обслуживания:

WHERE

```
CASE WHEN fare_condition = 'Business'
THEN seat_row::integer <= max_seat_row_business
WHEN fare_condition = 'Economy'
THEN seat_row::integer > max_seat_row_business
AND seat_row::integer <= max_seat_row_economy
```

В этом выражении используется приведение типов: `seat_row::integer`. Эта операция необходима, т. к. в виртуальной таблице номера рядов представлены в виде символьных строк, а для выполнения сравнения числовых значений в данной ситуации нужен целый тип. При написании условного оператора нужно учесть, что в виртуальной таблице мы указали не количество рядов в бизнес-классе и экономическом классе, а номера *последних* рядов в этих классах. Поэтому возникает конструкция

```
THEN seat_row::integer > max_seat_row_business
AND seat_row::integer <= max_seat_row_economy
```

Также проверяем еще одно условие, сравнивая символьные строки:

```
AND letter <= max_letter;
```

Последний этап в работе оператора SELECT — это формирование списка выражений, которые будут выведены в качестве итоговых данных. Для формирования номера места используется операция конкатенации `||`, которая соединяет номер ряда с буквенным обозначением позиции в ряду.

```
SELECT aircraft_code, seat_row || letter, fare_condition
```

Итак, SQL-команда, которая позволит за одну операцию ввести в таблицу «Места» сразу необходимое число строк, выглядит так:

```

INSERT INTO seats ( aircraft_code, seat_no, fare_conditions )
SELECT aircraft_code, seat_row || letter, fare_condition
FROM
    -- компоновки салонов
    ( VALUES ( 'SU9', 3, 20, 'F' ),
              ( '773', 5, 30, 'I' ),
              ( '763', 4, 25, 'H' ),
              ( '733', 3, 20, 'F' ),
              ( '320', 5, 25, 'F' ),
              ( '321', 4, 20, 'F' ),
              ( '319', 3, 20, 'F' ),
              ( 'CN1', 0, 10, 'B' ),
              ( 'CR2', 2, 15, 'D' )
    ) AS aircraft_info ( aircraft_code, max_seat_row_business,
                        max_seat_row_economy, max_letter )
CROSS JOIN
    -- классы обслуживания
    ( VALUES ( 'Business' ), ( 'Economy' )
    ) AS fare_conditions (
        fare_condition )
CROSS JOIN
    -- список номеров рядов кресел
    ( VALUES ( '1' ), ( '2' ), ( '3' ), ( '4' ), ( '5' ),
              ( '6' ), ( '7' ), ( '8' ), ( '9' ), ( '10' ),
              ( '11' ), ( '12' ), ( '13' ), ( '14' ), ( '15' ),
              ( '16' ), ( '17' ), ( '18' ), ( '19' ), ( '20' ),
              ( '21' ), ( '22' ), ( '23' ), ( '24' ), ( '25' ),
              ( '26' ), ( '27' ), ( '28' ), ( '29' ), ( '30' )
    ) AS seat_rows ( seat_row )
CROSS JOIN
    -- список номеров (позиций) кресел в ряду
    ( VALUES ( 'A' ), ( 'B' ), ( 'C' ), ( 'D' ), ( 'E' ),
              ( 'F' ), ( 'G' ), ( 'H' ), ( 'I' )
    ) AS letters ( letter )
WHERE
    CASE WHEN fare_condition = 'Business'
        THEN seat_row::integer <= max_seat_row_business
        WHEN fare_condition = 'Economy'
        THEN seat_row::integer > max_seat_row_business
        AND seat_row::integer <= max_seat_row_economy
    END
    AND letter <= max_letter;

```

Задание. Модифицируйте команду с учетом того, что в салоне бизнес-класса число мест в ряду должно быть меньше, чем в салоне экономического класса (в приведенном решении мы для упрощения задачи принимали эти числа одинаковыми).

Попробуйте упростить подзапрос, отвечающий за формирование списка номеров рядов кресел:

```
( VALUES ( '1' ), ( '2' ), ( '3' ), ( '4' ), ( '5' ), ...
```

Воспользуйтесь функцией `generate_series`, описанной в разделе документации 9.24 «Функции, возвращающие множества».